

Student name : Lubna Al Haani Binti Radzuan (A23CS0107)
Harini A/P Sangaran (A23CS0081)

DATABASE PROGRAMMING (SECP3623)
2025/2026 – SEM I

QUESTION 1: Library Management System Author

<u>author_id</u>	name
1	Haruki Murakami
2	Michelle Obama
3	Yuval Noah Harari

Book

<u>book_id</u>	title	genre	authorID
101	Kafka on the Shore	Fiction	1
101	Becoming	Biography	2
102	Sapeins	History	3
103	Norwegian Wood	Fiction	1

Loan

<u>loan_id</u>	<u>book_id</u>	member_name	loan_date	due_date	return_date
L10	101	Ali Ahmad	2025-09-01	2025-09-15	NULL
L11	101	Mei Ling	2025-09-03	2025-09-17	2025-09-10
L12	102	Ravi Kumar	2025-09-05	2025-09-19	NULL
L13	103	Aida Zulkifli	2025-09-06	2025-09-20	2025-09-18

Write SQL statements for each of the following questions:

- Create Book and Loan relations matching the structures shown.
- Rename authorID → author_id in Book relation.
- List book_id, title for books where genre = 'Fiction'.
- Find books with titles containing 'on'.
- Count how many books for each genre.
- Update a book title typo: 'Sapeins' to 'Sapiens'.
- Delete all loans that have a return_date.
- Insert a new author record.
- Make all NULL genres temporarily 'Unknown'.
- List all loans due between '2025-09-15' and '2025-09-19'.
- List books that have never been loaned.

Question 1

a) CREATE TABLE [IF NOT EXIST] Book(

book_id INT PRIMARY KEY,

title VARCHAR(255) NOT NULL,

genre VARCHAR(255),

authorID INT NOT NULL,

FOREIGN KEY (authorID)

REFERENCES Author (author_id)

ON UPDATE RESTRICT ON DELETE CASCADE

);

CREATE TABLE [IF NOT EXIST] Loan(

loan_id VARCHAR(255) PRIMARY KEY,

book_id INT,

member_name VARCHAR(255) NOT NULL,

loan_date DATE NOT NULL,

due_date DATE NOT NULL,

return_date DATE,

FOREIGN KEY (book_id)

REFERENCES Book (book_id)

ON UPDATE RESTRICT ON DELETE CASCADE

);

b) ALTER TABLE Book

CHANGE COLUMN authorID author_id INT NOT NULL;

c) SELECT book_id, title FROM Book WHERE genre = 'Fiction';

d) SELECT * FROM Book WHERE title LIKE '%on%';

e) SELECT genre COUNT(*) FROM Book GROUP BY genre;

f) UPDATE Book SET title = 'Sapiens' WHERE book_id = 102;

g) DELETE FROM Loan WHERE return_date IS NOT NULL;

h) INSERT INTO Author (author_id, name) VALUES (4, 'Haani');

i) UPDATE Book SET genre = 'Unknown' WHERE genre IS NULL;

j) SELECT * FROM Loan WHERE due_date BETWEEN '2025-09-15' AND '2025-09-19';

k) SELECT b.book_id, b.title FROM Book b LEFT JOIN Loan l ON b.book_id = l.book_id
WHERE l.book_id IS NULL;

QUESTION 2: Online Food Delivery

Restaurant (rest_id, rest_name, city) **PK: rest_id**

MenuItem (item_id, rest_id, item_name, price, category) **PK: item_id**

Orders (order_id, rest_id, customer, order_time, status) **PK: order_id**

OrderItem(order_id, item_id, qty) **PK: order_id, item_id**

Write SQL statements for each of the following questions:

- a) Add a UNIQUE constraint to prevent duplicate item names values within the same restaurant.
- b) Add CHECK constraints ensuring MenuItem.price > 0 and OrderItem.qty > 0.
- c) Retrieve all menu items whose item_name contains word 'Laksa'.
- d) Increase the price of all items in category 'Drink' by RM0.50.
- e) Update Orders.status to the enumerated set {'Preparing', 'Delivered', 'Cancelled'}.
- f) Identify the most expensive menu item for each restaurant.
- g) List all menu items that have never been ordered.
- h) Count the number of delivered orders for each restaurant.
- i) Compute the number of menu items offered by each restaurant.
- j) Retrieve item_id, item_name and price for the five most expensive items where category = 'Main', sorted by price descending.
- k) Delete any OrderItem rows with qty = 0.

Question 2

a) ALTER TABLE MenuItem

```
ADD CONSTRAINT unique_menu_item UNIQUE (item_id);
```

b) ALTER TABLE MenuItem

```
ADD CONSTRAINT check_price CHECK (price > 0);
```

ALTER TABLE OrderItem

```
ADD CONSTRAINT check_qty CHECK (qty > 0);
```

c) SELECT * FROM MenuItem WHERE item_name LIKE '%LAKSA%';

d) UPDATE MenuItem SET price = price + 0.50 WHERE category = 'Drink';

e) ALTER TABLE Orders

```
ADD CONSTRAINT check_order_status CHECK (status IN ('Preparing', 'Delivered',  
"Canceled"));
```

f) SELECT m.rest_id, r.rest_name, m.item_id, m.item_name, m.price

```
FROM MenuItem m
```

```
JOIN Restaurant r ON m.rest_id = r.rest_id
```

```
WHERE m.price = (SELECT MAX(price) FROM MenuItem WHERE m.rest_id = r.rest_id);
```

g) SELECT m.item_id, m.item_name, m.rest_id

```
FROM MenuItem m
```

```
LEFT JOIN OrderItem oi ON m.item_id = oi.item_id
```

```
WHERE oi.item_id IS NULL;
```

h) SELECT r.rest_id, r.rest_name, COUNT(o.order_id)

```
FROM Orders o
```

```
LEFT JOIN Restaurant r ON o.rest_id = r.rest_id
```

```
WHERE o.status = 'Delivered'
```

```
GROUP BY r.rest_id, r.rest_name;
```

i) SELECT r.rest_id, r.rest_name, m.rest_id COUNT(m.item_id)

```
FROM Restaurant r
```

```
LEFT JOIN MenuItem m ON r.item_id = m.rest_id
```

```
GROUP BY r.rest_id, r.rest_name;
```

j) SELECT item_id, item_name, price

```
FROM MenuItem
```

```
WHERE category = 'Main'
```

```
ORDER BY price DESC
```

```
LIMIT 5;
```

k) DELETE FROM OrderItem

```
WHERE qty = 0;
```

QUESTION 3: University Course Registration Student

<u>Sid</u>	Sname	Program
AC101	Azli Ismail	SECPH
AC102	Lee Mei Ling	SECRH
AC103	Vinesh A/L Ganesan	SECJH
AC104	Mardihah Kamal	SECVH

Course

<u>Cid</u>	Cname	Credit
SECP3623	Database Programming	3
SECR2414	Operating System	4
SECD2523	Database	3
SECI1143	Statistical Data Analysis	3
SECP2753	Data Mining	3

Enroll

<u>Sid</u>	<u>Cid</u>	<u>Semester</u>	Grade
AC101	SECP3623	202520261	A
AC101	SECI1143	202520261	B+
AC102	SECR2414	202520261	A-
AC102	SECD2523	202520261	A
AC103	SECI1143	202520261	A

Write SQL statements for each of the following questions:

- Create the Enroll table with a composite Primary Key (Sid, Cid, Semester) and Foreign Key constraints.
- Add a GPA DECIMAL(3,2) attribute to the Student table.
- Add a UNIQUE constraint on (Program, Sname) to prevent duplicate student names within the same program.
- Update Vinesh's program to 'SECPH'.
- Insert a new student and enroll the student in two courses within the same semester.
- Retrieve the course(s) that no student has enrolled in.
- List enroll records in semester '202520261' with grade A (any course).
- List students who have taken both 'Database Programming' and 'Database' courses.
- List (Sid, Sname) for students not enrolled in any course.
- Compute each student's total registered credits in semester 202520261.
- Delete enroll records for a student who withdraws (e.g. Sid = 'AC104').

Question 3

- a) CREATE TABLE [IF NOT EXIST] Enroll (
 Sid VARCHAR(10),
 Cid VARCHAR(10),
 Semester VARCHAR(15),
 Grade VARCHAR(5),
 PRIMARY KEY (Sid, Cid, Semester),
 FOREIGN KEY (Sid) REFERENCES Student(Sid),
 FOREIGN KEY (Cid) REFERENCES Course(Cid));
- b) ALTER TABLE Student
 ADD GPA DECIMAL(3, 2);
- c) ALTER TABLE Student
 ADD CONSTRAINT unique_program_sname UNIQUE (Program,
 Sname);
- d) UPDATE Student
 SET Program = 'SECPH'
 WHERE Sname = 'Vinesh A/L Ganesan';
- e) INSERT INTO Student (Sid, Sname, Program, GPA)
 VALUES ('AC105', 'Harini A/P Sangaran', 'SECPH', NULL);

 INSERT INTO Enroll (Sid, Cid, Semester, Grade)
 VALUES ('AC105', 'SECP3623', '202520261', NULL),
 ('AC105', 'SECP2753', '202520261', NULL);
- f) SELECT Cid, Cname
 FROM Course
 WHERE Cid NOT IN (SELECT Cid FROM Enroll);
- g) SELECT Sid, Cid, Grade
 FROM Enroll
 WHERE Semester = '202520261' AND Grade = 'A';
- h) SELECT e.Sid, s.Sname
 FROM Enroll e
 LEFT JOIN Student s ON e.Sid = s.Sid
 WHERE e.Cid IN ('SECP3623', 'SECD2523')
 GROUP BY e.Sid, s.Sname;

- i)

```
SELECT s.Sid, s.Sname
FROM Student s
LEFT JOIN Enroll e ON s.Sid = e.Sid
WHERE e.Sid IS NULL;
```
- j)

```
SELECT e.Sid, s.Sname, SUM(c.Credit)
FROM Enroll e
JOIN Student s ON e.Sid = s.Sid
JOIN Course c ON e.Cid = c.Cid
WHERE e.Semester = '202520261'
GROUP BY e.Sid, s.Sname;
```
- k)

```
DELETE FROM Student
WHERE Sid = 'AC104';
```